

Guide du Projet — 3S TalentMatch

Document de référence technique et fonctionnel

Table des matières

- 1. [Vue d'ensemble](#)
- 2. [Architecture technique](#)
- 3. [Base de données — Tables & Relations](#)
- 4. [Authentification & Rôles](#)
- 5. [Upload & Extraction de CV](#)
- 6. [Parsing NLP — Analyse du CV](#)
- 7. [Gestion des offres d'emploi](#)
- 8. [Moteur de Matching IA](#)
- 9. [Dashboard & Statistiques](#)
- 10. [Notifications](#)
- 11. [RGPD & Conformité](#)
- 12. [Frontend — Interface utilisateur](#)

1. Vue d'ensemble

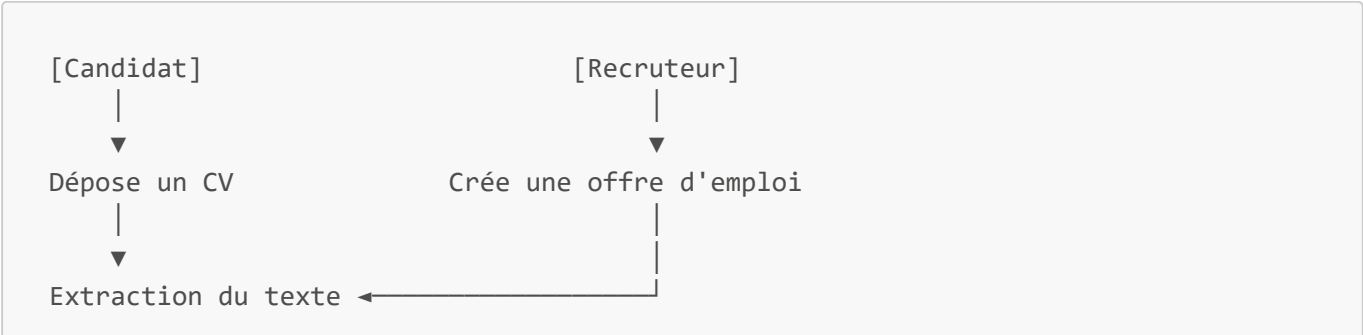
C'est quoi ?

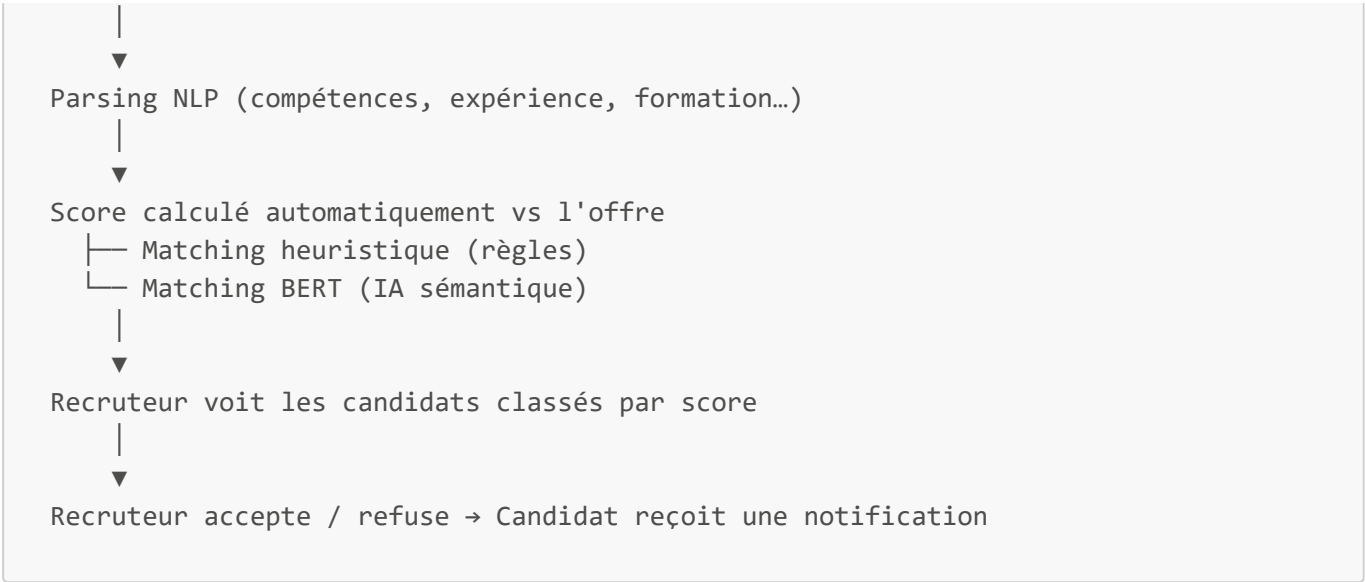
3S TalentMatch est une plateforme web de recrutement intelligente. Elle permet à des candidats de déposer leur CV et à des recruteurs de trouver automatiquement les meilleurs profils pour leurs offres grâce à l'intelligence artificielle.

Les 3 rôles utilisateurs

Rôle	Ce qu'il peut faire
Candidat	S'inscrire, déposer son CV, consulter les offres, suivre ses candidatures
Recruteur	Gérer les offres, consulter les candidats classés par l'IA, changer les statuts
Administrateur	Gérer tous les utilisateurs, créer des recruteurs, consulter les logs RGPD

Le flux principal





2. Architecture technique

Stack technologique

Couche	Technologie	Rôle
Backend	FastAPI (Python)	API REST, logique métier, IA
Base de données	PostgreSQL	Stockage des données
ORM & Migrations	SQLAlchemy + Alembic	Modèles de données et versioning BDD
Frontend	React + Vite	Interface utilisateur
Authentification	JWT (python-jose)	Sécurité des endpoints
NLP	spaCy (fr_core_news_md)	Analyse de texte en français
IA Matching	sentence-transformers (BERT)	Similarité sémantique
Email	SMTP Gmail	Notifications par email

Structure des dossiers



— components/	← Composants réutilisables
— services/api.js	← Appels API vers le backend
— AuthContext.jsx	← Gestion de la session utilisateur

Comment les couches communiquent

```

Navigateur (React)
  | requête HTTP + JWT token
  ▼
Backend FastAPI (port 8000)
  | lecture/écriture
  ▼
Base de données PostgreSQL

```

Le frontend ne parle JAMAIS directement à la base de données. Tout passe par l'API backend.

3. Base de données — Tables & Relations

Fichiers concernés

- `backend/app/models/user.py` — table `users`
- `backend/app/models/candidate.py` — table `candidates`
- `backend/app/models/cv_document.py` — table `cv_documents`
- `backend/app/models/job_offer.py` — tables `job_offers` + `offer_recruiters`
- `backend/app/models/match.py` — table `matches`
- `backend/app/models/notification.py` — table `notifications`
- `backend/app/models/access_log.py` — table `access_logs`
- `backend/alembic/versions/` — 9 fichiers de migration

Technologie utilisée

Outil	Rôle
PostgreSQL	Base de données relationnelle (données stockées sur disque)
SQLAlchemy	ORM Python — permet d'écrire des requêtes en Python, pas en SQL
Alembic	Gestion des migrations (versioning du schéma de BDD)

Qu'est-ce qu'un ORM ? Sans ORM, pour récupérer un utilisateur on écrit :

```
SELECT * FROM users WHERE email = 'youssef@gmail.com';
```

Avec SQLAlchemy (ORM), on écrit directement en Python :

```
user = db.query(User).filter(User.email == "youssef@gmail.com").first()
```

L'ORM traduit automatiquement le code Python en SQL. Plus besoin d'écrire du SQL à la main.

Les 8 tables de la base de données

Table **users** — Tous les utilisateurs

users	
id (UUID, PK)	Identifiant unique
nom	Nom de famille
prenom	Prénom
email	Email (unique, indexé)
hashed_password	Mot de passe hashé (bcrypt)
role	candidat / recruteur / admin
is_active	Compte actif ou désactivé
is_email_verified	Email vérifié via OTP ?
verification_code	Code OTP envoyé par email
reset_code	Code de réinit. mot de passe
auth_provider	local / google / linkedin
oauth_id	ID Google ou LinkedIn
avatar_url	Photo de profil OAuth
created_at	Date de création

Table **candidates** — Profils CV extraits

candidates	
id (UUID, PK)	Identifiant unique
cv_id	Identifiant du fichier CV
user_id (FK→users)	Qui a uploadé ce CV
nom, email, tel	Infos extraites du CV
parsed_data (JSON)	Tout le résultat du NLP
raw_text	Texte brut extrait
extraction_method	pypdf / ocr / docx
candidature_status	en_attente / accepte / refuse
anonymized	RGPD : données effacées ?
created_at	Date d'upload

Note importante : `parsed_data` est un champ JSON — il stocke tout le résultat du parsing NLP (compétences, expériences, formations, langues...) dans une seule colonne. Ça évite des dizaines de tables.

Table `cv_documents` — Fichiers uploadés

cv_documents	
id (UUID, PK)	Identifiant unique
candidate_id (FK→candid.)	À quel candidat
filename	Nom du fichier
file_type	pdf / docx / image
extraction_method	pypdf / ocr / docx
raw_text	Texte extrait
status	uploaded/extracted/parsed

Table `job_offers` — Offres d'emploi

job_offers	
id (UUID, PK)	Identifiant unique
recruiter_id (FK→users)	Créateur de l'offre
titre	Ex: "Développeur Python"
description	Texte complet de l'offre
competences_requises	JSON: ["Python", "React"]
localisation	Ex: "Tunis"
type_contrat	CDI / CDD / Stage...
nb_postes	Nombre de postes
status	active / closed / draft
date_limite	Date de clôture

Table `offer_recruiters` — Recruteurs assignés aux offres

C'est une **table de jonction** (Many-to-Many) : une offre peut avoir plusieurs recruteurs, un recruteur peut gérer plusieurs offres.

offer_recruiters	
offer_id (FK)	→ job_offers.id
user_id (FK)	→ users.id

Table **matches** — Résultats de matching

matches	
id (UUID, PK)	Identifiant unique
candidate_id (FK)	→ candidates.id
job_offer_id (FK)	→ job_offers.id
score (Float)	Score 0.0 à 1.0 (0%→100%)
details (Text/JSON)	Détail par dimension
status	pending/reviewed/accepted
created_at	Date du calcul

Chaque fois qu'un CV est uploadé, une ligne **matches** est créée automatiquement pour **chaque offre active**. C'est ce qui permet d'afficher le classement des candidats instantanément.

Table **notifications** — Notifications utilisateur

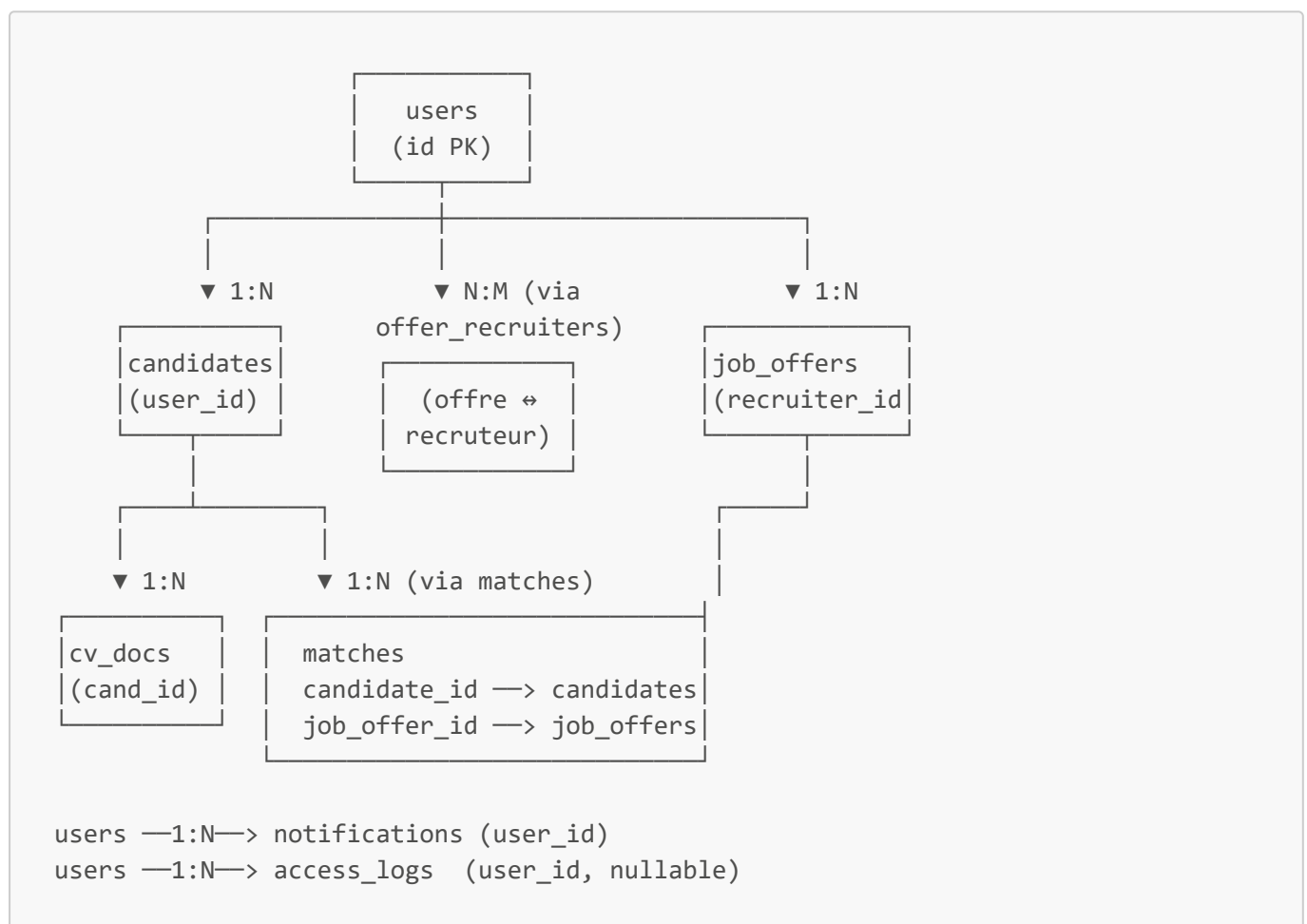
notifications	
id (UUID, PK)	Identifiant unique
user_id (FK→users)	Destinataire
type	status_change / new_cv
title	Titre de la notification
message	Contenu
link	URL vers la page concernée
is_read	Lu ou non
created_at	Date

Table **access_logs** — Journal RGPD

access_logs	
-------------	--

id (UUID, PK)	Identifiant unique
user_id (nullable)	Qui a fait l'action
user_email	Email (archivé)
user_role	Rôle au moment de l'action
action	UPLOAD_CV / VIEW_CANDIDATE
resource_type	candidate / offer / match
resource_id	ID de la ressource
detail	Informations supplémentaires
ip_address	Adresse IP
created_at	Date et heure exacte

Diagramme des relations entre tables



Lecture du diagramme :

- **1:N** = un utilisateur peut avoir plusieurs candidats (one-to-many)
- **N:M** = une offre peut avoir plusieurs recruteurs ET un recruteur peut gérer plusieurs offres (many-to-many, via la table **offer_recruiters**)

Pourquoi UUID et pas des IDs numériques (1, 2, 3...) ?

Les IDs sont des **UUID** (ex: **a3f7c2b1-4d5e-...**) au lieu de simples nombres.

Raisons :

1. **Sécurité** — on ne peut pas deviner l'ID d'un autre utilisateur (`/users/3` → `/users/4`)
2. **Scalabilité** — si on fusionne deux bases de données, pas de collision d'IDs
3. **RGPD** — les IDs ne révèlent pas d'information sur l'ordre de création

Les migrations Alembic — comment le schéma a évolué

Qu'est-ce qu'une migration ? Chaque fois qu'on modifie la structure de la base de données (ajouter une colonne, créer une table...), on crée un fichier de migration. Alembic applique ces fichiers dans l'ordre chronologique.

Historique des 9 migrations du projet :

#	Fichier	Ce qui a été ajouté
1	<code>87ddcf...</code>	Table <code>candidates</code> (version initiale)
2	<code>ee3700...</code>	Tables <code>users</code> , <code>cv_documents</code> , <code>job_offers</code> , <code>matches</code>
3	<code>50b7de...</code>	Champs auth dans <code>users</code> (<code>hashed_password</code> , <code>auth_provider</code> , <code>oauth_id</code>)
4	<code>0d5aca...</code>	<code>user_id</code> dans <code>candidates</code> , <code>candidature_status</code>
5	<code>6c5981...</code>	Vérification email + reset mot de passe (<code>verification_code</code> , <code>reset_code</code>)
6	<code>a1b2c3...</code>	Table <code>access_logs</code> , champs RGPD dans <code>candidates</code>
7	<code>b2c3d4...</code>	Table <code>notifications</code>
8	<code>c3d4e5...</code>	Table <code>offer_recruiters</code> (recruteurs assignés)
9	<code>d4e5f6...</code>	Champ <code>date_limite</code> dans <code>job_offers</code>

Comment appliquer toutes les migrations :

```
alembic upgrade head
```

Cette commande s'exécute automatiquement au démarrage du serveur sur Railway.

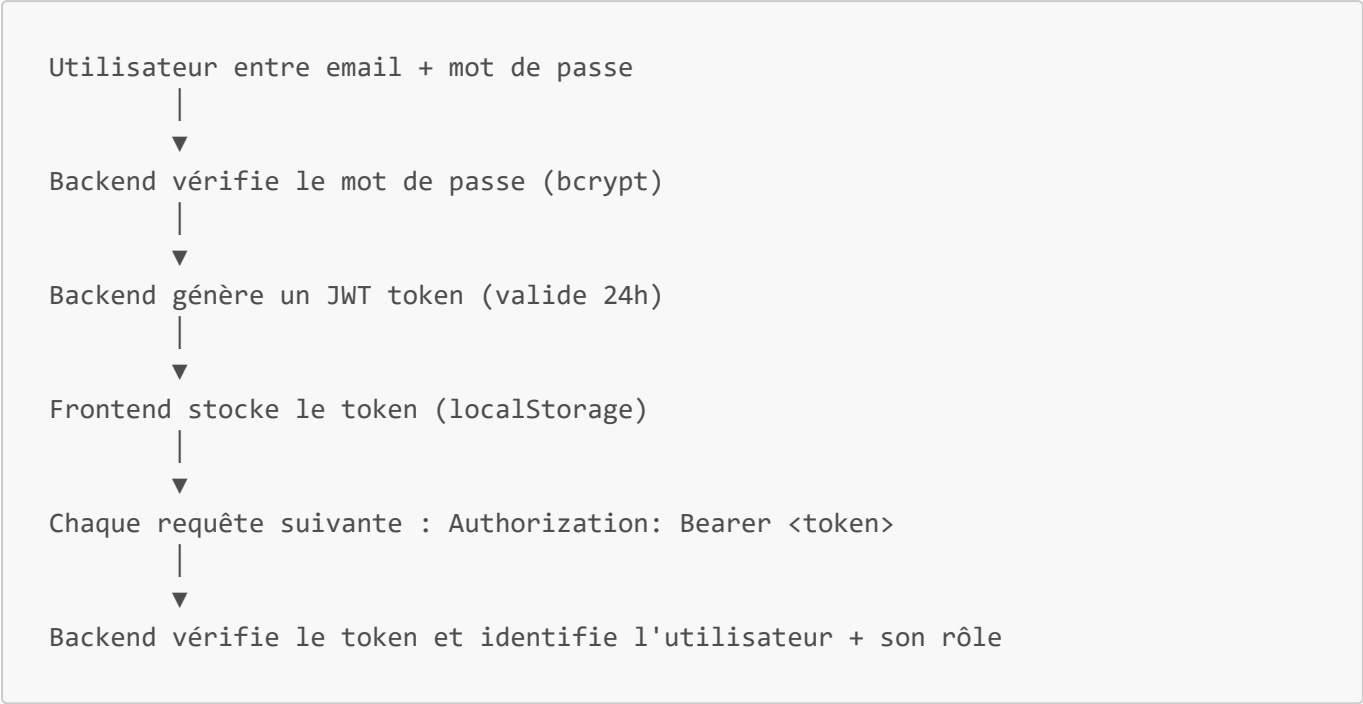
4. Authentification & Rôles

Fichiers concernés

- `backend/app/routes/auth.py` — tous les endpoints d'authentification
- `backend/app/models/user.py` — modèle utilisateur
- `frontend/src/AuthContext.jsx` — gestion du token côté React
- `frontend/src/pages/Login.jsx` — page de connexion

Comment ça marche

L'authentification utilise **JWT (JSON Web Token)**. Quand un utilisateur se connecte, le serveur génère un token signé que le frontend stocke et envoie à chaque requête.



Exemple — Contenu d'un token JWT décodé

```
{
  "sub": "42",
  "email": "youssef@example.com",
  "role": "recruteur",
  "exp": 1716500000
}
```

Endpoints disponibles

Endpoint	Méthode	Description
/api/auth/register	POST	Inscription avec email/mot de passe
/api/auth/login	POST	Connexion → retourne le JWT
/api/auth/verify-email	POST	Vérification du code OTP reçu par email
/api/auth/forgot-password	POST	Envoie un code de réinitialisation
/api/auth/reset-password	POST	Réinitialise le mot de passe
/api/auth/oauth/google	POST	Connexion via Google OAuth
/api/auth/me	GET	Retourne le profil de l'utilisateur connecté

Vérification email (OTP)

Lors de l'inscription, un code à 6 chiffres est envoyé par email. L'utilisateur a **15 minutes** pour le saisir. Sans vérification, il ne peut pas se connecter.

```
# backend/app/routes/auth.py
# Code OTP généré aléatoirement
verification_code = str(random.randint(100000, 999999))
```

Contrôle des accès par rôle

Chaque endpoint est protégé par une dépendance FastAPI :

```
# Seulement les recruteurs et admins
async def get_current_recruteur_or_admin(user = Depends(get_current_user)):
    if user.role not in ["recruteur", "admin"]:
        raise HTTPException(403, "Accès refusé")
    return user
```

5. Upload & Extraction de CV

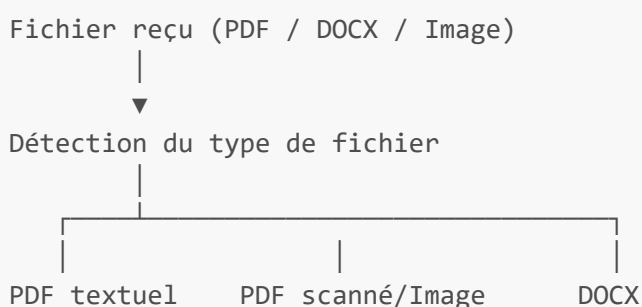
Fichiers concernés

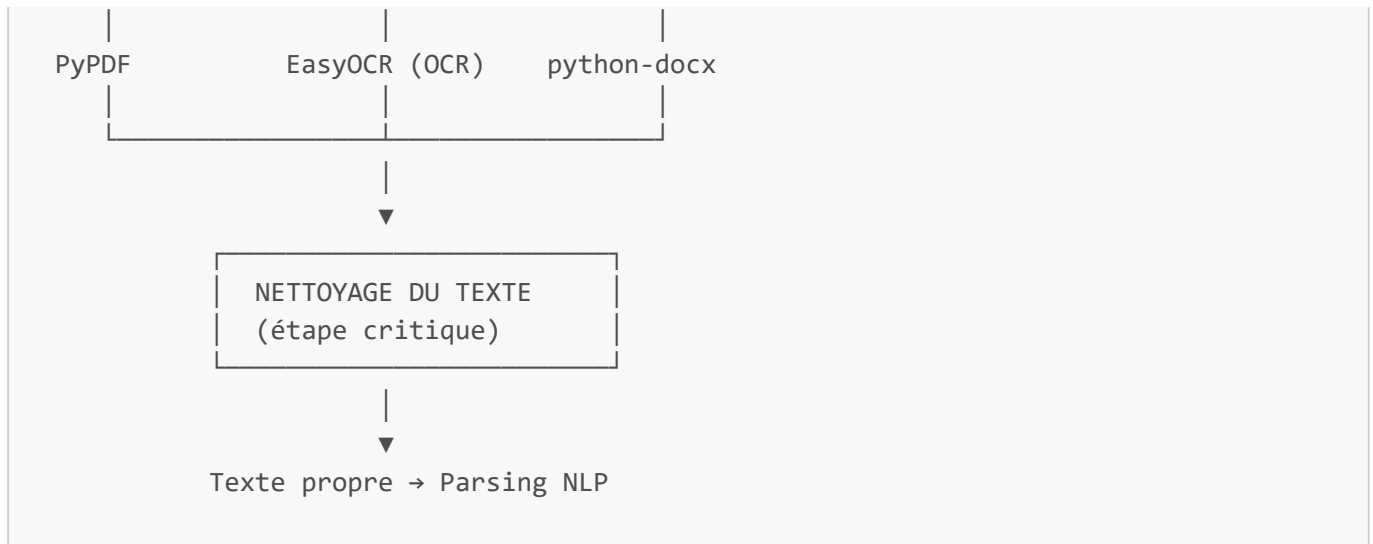
- `backend/app/routes/cv.py` — endpoint d'upload
- `backend/app/services/nlp/nlp_parser.py` — nettoyage et orchestration
- `backend/app/services/nlp/skills_extractor.py` — extraction compétences
- `backend/app/services/nlp/experience_extractor.py` — extraction expériences
- `backend/app/services/nlp/formation_extractor.py` — extraction formations
- `backend/app/services/nlp/contact_extractor.py` — extraction contacts

Formats acceptés

- **PDF textuel** → extraction directe avec PyPDF
- **PDF scanné / Image (PNG, JPG)** → OCR avec EasyOCR
- **DOCX** → extraction avec python-docx
- Taille maximale : **10 Mo**

Le processus d'extraction étape par étape





Pourquoi une étape de nettoyage ? (c'est important)

Les PDFs sont souvent mal exportés. Voici des problèmes réels que le code corrige automatiquement :

Problème 1 — Mots cassés par le PDF

```
PDF brut : "Djang o"   "Pytorc h"   "TensorFlo w"
Après fix : "Django"   "PyTorch"   "TensorFlow"
```

Le code a un dictionnaire de ~100 mots cassés courants dans `nlp_parser.py`.

Problème 2 — Accents corrompus

```
PDF brut : "´ e"   "` e"   "^ a"
Après fix : "é"   "è"   "â"
```

Les PDFs stockent parfois les accents comme deux caractères séparés.

Problème 3 — Tout sur une seule ligne (PDF compact)

```
PDF brut : "Python Java React EXPÉRIENCES Développeur chez TechCorp 2022-2024
FORMATIONS..."
Après fix : sections bien séparées avec des sauts de ligne
```

Le code détecte les mots-clés de sections (EXPÉRIENCES, FORMATIONS, COMPÉTENCES...) et réinjecte des retours à la ligne.

Problème 4 — Texte CamelCase collé

```
PDF brut : "2022-2024Senior Developer"
```

```
Après fix : "2022-2024\nSenior Developer"
```

Ce qui se passe après l'upload

1. Le fichier original est sauvegardé dans `data/cvs_raw/originals/`
2. Le texte nettoyé est analysé par le NLP (section suivante)
3. Les données parsées sont stockées en base de données (colonne `parsed_data` en JSON)
4. Un score de matching est calculé automatiquement pour **chaque offre active**
5. Les recruteurs assignés reçoivent une **notification**

6. Parsing NLP — Analyse du CV

Fichiers concernés

- `backend/app/services/nlp/nlp_parser.py` — orchestrateur du pipeline
- `backend/app/services/nlp/skills_extractor.py` — extraction compétences (300+ termes)
- `backend/app/services/nlp/experience_extractor.py` — extraction expériences
- `backend/app/services/nlp/formation_extractor.py` — extraction formations
- `backend/app/services/nlp/contact_extractor.py` — emails, téléphones, LinkedIn
- `backend/app/services/nlp/entity_extractor.py` — NER spaCy (noms de personnes)
- `backend/app/services/nlp/sector_classifier.py` — classification sectorielle
- `backend/app/models/candidate.py` — modèle candidat avec `parsed_data`

Qu'est-ce que le NLP ici ?

NLP = Natural Language Processing (Traitement Automatique du Langage). On utilise **spaCy** avec le modèle français `fr_core_news_md` — un modèle pré-entraîné sur des millions de textes français, capable de reconnaître des entités (noms de personnes, organisations, dates) dans un texte.

Le pipeline en 6 étapes (dans l'ordre d'exécution)

```
Texte nettoyé du CV
|
▼
[Étape 1] CONTACTS
→ Regex pour email, téléphone, LinkedIn, GitHub
→ Exemple: "youssef@gmail.com" → { email: "youssef@gmail.com" }
|
▼
[Étape 2] NOM DE LA PERSONNE
→ spaCy NER (reconnaissance d'entités PERSON)
→ Fallback: CamemBERT (si configuré) ou dérivé depuis l'email
→ Si spaCy et CamemBERT sont d'accord → confiance maximale
|
▼
[Étape 3] COMPÉTENCES (le plus important)
→ Dictionnaire de 300+ compétences techniques
→ RapidFuzz pour corriger les fautes (voir ci-dessous)
```

→ spaCy NER découvre des compétences inconnues dans le texte



[Étape 4] FORMATIONS

- Patterns regex pour diplômes (Licence, Master, Bac+X...)
- spaCy NER reconnaît les noms d'établissements (ORG)
- Détecte les années (DATE)



[Étape 5] EXPÉRIENCES

- Patterns regex pour titres de postes (Développeur, Ingénieur...)
- spaCy NER reconnaît les entreprises (ORG)
- Calcule la durée totale en mois



[Étape 6] LANGUES

- Regex sur les sections "Langues / Languages"
- Détecte la langue + le niveau (Courant, B2, Bilingue...)

Comment fonctionne la correction de fautes ? (RapidFuzz)

L'outil utilisé est **RapidFuzz** (`from rapidfuzz import fuzz`). Ce n'est pas un correcteur orthographique classique — c'est un moteur de **similarité floue** entre chaînes de caractères.

Principe :

```
fuzz.token_set_ratio("pydon", "python") → 83%   ✓ (≥ 80% : accepté)
fuzz.token_set_ratio("reactjs", "react") → 86%   ✓ (accepté)
fuzz.token_set_ratio("java", "javascript") → 62%  X (< 80% : refusé)
```

Le seuil est fixé à **80%** : si deux mots sont similaires à 80% ou plus, on considère que c'est la même compétence.

En plus de RapidFuzz, il y a un dictionnaire d'alias (`_SKILL_ALIASES` dans `match_engine.py`) qui mappe les variantes connues vers la forme canonique :

```
# Exemples du dictionnaire (environ 100 entrées)
"js"      → "javascript"
"reactjs" → "react"
"react.js" → "react"
"py"      → "python"
"python3" → "python"
"sklearn" → "scikit-learn"
"k8s"     → "kubernetes"
"postgres" → "postgresql"
"nodejs"  → "node.js"
"tf"      → "tensorflow"
"dl"      → "deep learning"
"ml"      → "machine learning"
```

```
"scrum"    → "agile"  
"github"   → "git"
```

Résultat : Une offre qui demande "React" et un CV qui écrit "ReactJS" → match parfait grâce à l'alias. Un CV avec "pydon" (faute de frappe) → RapidFuzz donne 83% de similarité avec "python" → accepté.

Ce qu'on extrait du CV — résultat final (JSON)

```
{  
  "identite": { "nom_complet": "Youssef Gara" },  
  "contacts": {  
    "email": "youssef@gmail.com",  
    "telephone": "+216 12 345 678",  
    "linkedin": "linkedin.com/in/youssef-gara"  
  },  
  "competences": [  
    { "name": "Python", "category": "Backend", "source": "dictionnaire" },  
    { "name": "React", "category": "Frontend", "source": "dictionnaire" },  
    { "name": "FastAPI", "category": "Backend", "source": "fuzzy" }  
  ],  
  "formations": [  
    {  
      "diplome": "Licence en Informatique",  
      "etablissement": "ESPRIT",  
      "annee": 2024,  
      "niveau_bac": 3  
    }  
  ],  
  "experiences": [  
    {  
      "poste": "Développeur Full Stack",  
      "entreprise": "3S",  
      "date_debut": "2026-02",  
      "duree_mois": 3,  
      "missions": ["Développement API REST", "Intégration NLP"]  
    }  
  ],  
  "langues": [  
    { "langue": "Français", "niveau": "Langue maternelle" },  
    { "langue": "Anglais", "niveau": "Courant" }  
  ],  
  "secteur_detecte": {  
    "secteur": "informatique",  
    "label": "Développement logiciel",  
    "confiance": 0.92  
  },  
  "metadata": {  
    "annees_experience_totales": 0.25,  
    "niveau_seniorite": "Junior",  
    "confidence_score": 0.84  
  }  
}
```

```
}  
}
```

7. Gestion des offres d'emploi

Fichiers concernés

- `backend/app/routes/job_offers.py` — endpoints offres
- `backend/app/models/job_offer.py` — modèle offre
- `frontend/src/pages/OffersList.jsx` — liste des offres
- `frontend/src/pages/OfferNew.jsx` — création d'offre
- `frontend/src/pages/OfferDetail.jsx` — détail d'une offre

Structure d'une offre

```
{  
  "titre": "Développeur Backend Python",  
  "description": "Nous recherchons un développeur...",  
  "competences_requises": ["Python", "FastAPI", "PostgreSQL"],  
  "localisation": "Tunis",  
  "type_contrat": "CDI",  
  "nb_postes": 2,  
  "date_limite": "2026-06-30",  
  "status": "active"  
}
```

Statuts d'une offre

```
draft → active → closed  
|  
|  
| — se ferme automatiquement si :  
|   - date limite dépassée  
|   - nombre de postes atteint  
| — brouillon non publié
```

Assignment des recruteurs

Un admin peut assigner plusieurs recruteurs à une même offre. Un recruteur ne voit que les offres qui lui sont assignées.

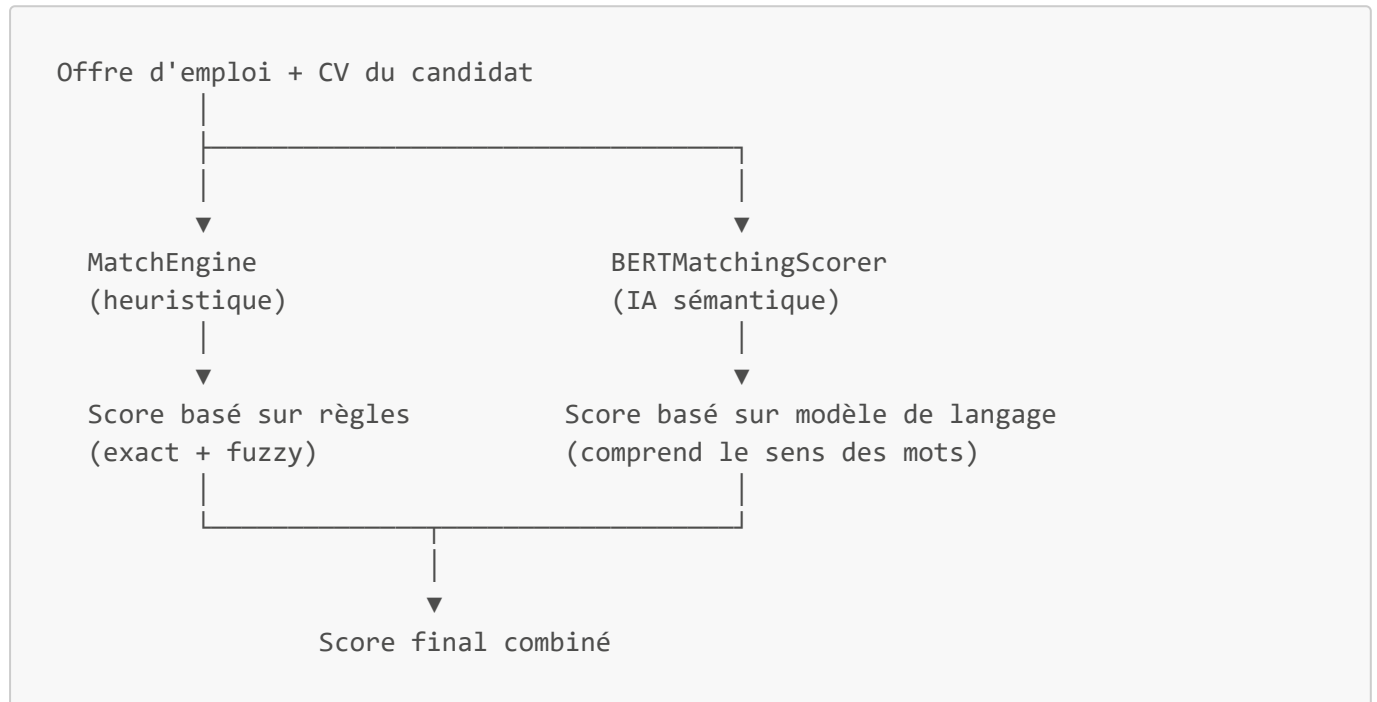
```
Offre "Dev Backend"  
├─ Recruteur A (peut voir les candidats)  
└─ Recruteur B (peut voir les candidats)
```

8. Moteur de Matching IA

Fichiers concernés

- `backend/app/services/matching/match_engine.py` — matching heuristique (MatchEngine)
- `backend/app/services/matching_sandbox/bert_scorer.py` — matching BERT (BERTMatchingScorer)
- `backend/app/routes/matching.py` — endpoints de matching

Vue d'ensemble : deux moteurs, un score final



Moteur 1 — MatchEngine (heuristique) — `match_engine.py`

Basé sur des règles précises et mesurables. 5 dimensions :

Dimension	Poids	Comment c'est calculé
Compétences	45%	Alias exact + RapidFuzz $\geq$ 80%
Expérience	25%	$\text{années_candidat} \div \text{années_requis}$
Formation	20%	$\text{niveau Bac+X candidat} \div \text{niveau requis}$
Localisation	10%	ville de l'offre présente dans le texte du CV
Sémantique	12%	similarité cosinus spaCy (vecteurs de mots)

Comment le score compétences est calculé exactement :

Offre demande : ["Python", "FastAPI", "React", "Docker"]

Pour chaque compétence requise :

1. Normalisation via alias dict ("ReactJS" → "react")
2. Comparaison exacte avec les compétences du CV
3. Si pas exact → RapidFuzz token_set_ratio
 - ratio ≥ 80% → considéré comme match
 - ratio < 80% → pas de match

Résultat :

```
Python → trouvé exact      → 1.0
FastAPI → trouvé fuzzy (91%) → 1.0
React   → non trouvé        → 0.0
Docker  → trouvé exact      → 1.0
```

Score compétences = $(1.0 + 1.0 + 0.0 + 1.0) / 4 = 0.75 \rightarrow 75\%$

Comment le score expérience est calculé :

```
# L'offre dit "3 ans d'expérience minimum"
# Le CV a 2 ans d'expérience
exp_score = min(1.0, 2 / 3) = 0.67 → 67%

# L'offre dit "2 ans minimum", CV a 5 ans
exp_score = min(1.0, 5 / 2) = 1.0 → 100% (plafonné)

# L'offre ne précise pas d'expérience
exp_score = 0.5 → score neutre
```

Comment la localisation est vérifiée :

```
# Offre localisation : "Tunis"
# Le système cherche "tunis" dans le texte brut du CV
# Si trouvé → 1.0 (100%), sinon → 0.0 (0%)
```

Moteur 2 — BERTMatchingScorer (IA) — [bert_scorer.py](#)

Utilise un **modèle de langage BERT** pour comprendre le sens des mots, pas seulement les mots exacts.

Quel modèle IA est utilisé ?

```
Priorité 1 → TalentMatch-BERT (modèle fine-tuné spécialement pour ce projet)
              Chemin : data/models/talentmatch-bert/
              Fine-tuné sur des paires CV / offres d'emploi

Priorité 2 → paraphrase-multilingual-MiniLM-L12-v2
              Modèle généraliste multilingue de HuggingFace
              Utilisé si TalentMatch-BERT n'est pas disponible
```

Comment BERT calcule la similarité ?

BERT transforme chaque texte en un **vecteur numérique** (embedding) dans un espace à 384 dimensions. Deux textes similaires ont des vecteurs proches.

Texte CV : "développement web, JavaScript, Node.js, 5 ans"

↓ encodé par BERT

[0.23, -0.45, 0.12, ..., 0.87] (384 nombres)

Texte Offre : "React developer, frontend, JavaScript requis"

↓ encodé par BERT

[0.25, -0.41, 0.15, ..., 0.82] (384 nombres)

Similarité cosinus = 0.78 → 78%

(vecteurs proches = textes sémantiquement proches)

Les 4 dimensions BERT (dans BERTMatchingScorer)

Dimension	Méthode	Ce que ça mesure
Compétences	Alias + RapidFuzz + bonus BERT (max +10%)	Compétences listées dans le CV
Expérience	50% ratio années + 50% similarité BERT du domaine	Domaine ET durée d'expérience
Formation	Ratio niveau Bac+X	Niveau de diplôme
Sémantique	Similarité cosinus BERT profil complet	Cohérence globale CV / offre

Les poids sont DYNAMIQUES — ils changent selon l'offre

C'est une fonctionnalité clé du système. Les poids s'adaptent automatiquement :

Offre stage / alternance :

Expérience → 3% (étudiant = pas d'expérience, c'est normal)

Compétences → 42%

Formation → 22% (le diplôme compte plus)

Sémantique → 33%

Offre junior (0-2 ans) :

Expérience → 6%

Compétences → 44%

Formation → 15%

Sémantique → 35%

Offre senior (5+ ans) :

Expérience → 33% (l'expérience est le critère principal)

Compétences → 34%

Formation → 13%

Sémantique → 20%

```
Poste technique (développeur) :  
  Compétences += 5% (les skills techniques priment)  
  
Poste management / commercial :  
  Sémantique += 8% (le profil global prime)
```

Plafonds et planchers du score final

```
# Plafond progressif selon les compétences matchées :  
compétences < 25% → score max 32% (hors domaine total)  
compétences < 40% → score max 42% (très peu de compétences)  
compétences ≤ 50% → score max 52% (match partiel)  
  
# Limites absolues :  
score minimum = 20% (tout CV soumis a au moins 20%)  
score maximum = 95% (aucun CV n'est parfait)
```

Pourquoi ces plafonds ? Un candidat avec un excellent BERT sémantique mais aucune compétence technique ne doit pas obtenir un score élevé. Le matching compétences reste le critère discriminant principal.

Détection de négation — une feature importante

Le parseur NLP peut extraire "Python" d'un CV qui dit "Aucune compétence en Python". Le scorer BERT détecte ça et exclut cette compétence du calcul.

```
# Contexte dans le texte brut du CV (60 caractères avant "python")  
# "...aucune expérience en python..."  
#      ↑ mot de négation détecté → "python" ignoré  
  
# Mots de négation reconnus :  
# "pas de", "sans", "aucune", "jamais", "no", "not", "without"  
# "peu d'exp", "n'ai pas", "ne pas", etc.
```

Détection d'inconsistances — signal pour le recruteur

Le système signale automatiquement les incohérences dans le CV :

Niveau	Problème	Exemple
1	Compétence absente du texte brut	"React" listé mais mot "react" introuvable dans le CV
2	Compétence absente des expériences	"Django" listé mais jamais mentionné dans les postes
3	Écosystème manquant	"React" listé mais pas de "JavaScript" dans le CV
4	Faible similarité BERT	Compétence très éloignée du contexte des expériences

Ce que le recruteur voit dans l'interface

Candidat : Youssef Gara
Score global : 78%

Dimensions :

Compétences		82%	(poids 42%)
Expérience		60%	(poids 20%)
Formation		80%	(poids 15%)
Sémantique		74%	(poids 23%)

Compétences matchées :

✓ Python (exact)

✓ FastAPI (fuzzy 91%)

✓ React (exact)

✗ Docker (non trouvé)

⚠ Inconsistances détectées :

"Django" listé mais absent des expériences

"AWS" mentionné sans contexte professionnel lié

9. Dashboard & Statistiques

Fichiers concernés

- `backend/app/routes/dashboard.py` — endpoint stats
- `frontend/src/pages/Dashboard.jsx` — interface dashboard

Ce que le dashboard affiche

KPIs principaux

24 CVs
reçus

5 Offres
actives

Score
moy: 71%

Taux
acc:
35%

Alertes

⚠ 3 CVs sans matching

⚠ 5 décisions en attente

Top candidats

1. Youssef G. — 91%

2. Asma B. — 87%

Pipeline par offre

Dev Backend : 12 candidats

UX Designer : 8 candidats

Activité 7 derniers jours

Lun: 3 CVs, Mar: 5 CVs

Données disponibles

- Totaux : candidats, offres, matchings lancés
- Distribution des scores : excellent (>75%) / moyen (50-75%) / faible (<50%)
- Statuts des candidatures : en attente / accepté / refusé
- Top 8 candidats par score
- Activité des 7 derniers jours
- Alertes automatiques (CVs sans score, décisions en attente...)

10. Notifications

Fichiers concernés

- `backend/app/models/notification.py` — modèle notification
- `backend/app/routes/notifications.py` — endpoints
- `frontend/src/components/NotificationBell.jsx` — cloche dans la navbar

Quand une notification est créée ?

Événement	Qui reçoit
Un candidat dépose un CV	Les recruteurs assignés à l'offre
Un recruteur change le statut (accepté/refusé)	Le candidat concerné

Exemple de notification

```
{
  "type": "status_change",
  "message": "Votre candidature pour 'Développeur Backend' a été acceptée.",
  "lien": "/my-applications",
  "lue": false,
  "created_at": "2026-05-12T10:30:00"
}
```

Email automatique

En plus de la notification dans l'interface, un **email** est envoyé automatiquement via Gmail SMTP au candidat quand son statut change.

11. RGPD & Conformité

Fichiers concernés

- `backend/app/models/access_log.py` — logs d'accès
- `backend/app/services/access_logger.py` — enregistrement des actions
- `backend/app/routes/admin.py` — consultation des logs (admin)

Principe

Toute action sur les données personnelles est **tracée** automatiquement pour se conformer au RGPD (Règlement Général sur la Protection des Données).

Actions tracées

Action	Description
UPLOAD_CV	Un CV est déposé
VIEW_CANDIDATE	Un recruteur consulte un profil
DOWNLOAD_CV	Un CV original est téléchargé
STATUS_CHANGED	Statut de candidature modifié
ANONYMIZE_CANDIDATE	Données personnelles anonymisées
DELETE_CANDIDATE	Candidat supprimé
MATCH_LAUNCHED	Matching IA lancé

Anonymisation

Un recruteur ou admin peut **anonymiser** un candidat : les données personnelles (nom, email, téléphone) sont remplacées par des valeurs neutres, mais le profil de compétences reste disponible pour les stats.

```
# backend/app/routes/cv.py
candidate.nom = "Anonymisé"
candidate.email = None
candidate.telephone = None
candidate.is_anonymized = True
```

12. Frontend — Interface utilisateur

Fichiers concernés

- `frontend/src/App.jsx` — définition de toutes les routes
- `frontend/src/services/api.js` — tous les appels API
- `frontend/src/pages/` — 20 pages React
- `frontend/src/AuthContext.jsx` — session utilisateur

Pages par rôle

Candidat

Page	Fichier	Description
Accueil personnalisé	<code>Home.jsx</code>	Résumé de ses candidatures

Page	Fichier	Description
Offres disponibles	<code>CandidateOffers.jsx</code>	Liste des offres actives
Détail d'une offre	<code>CandidateOfferDetail.jsx</code>	Voir une offre + postuler
Mes candidatures	<code>MyApplications.jsx</code>	Statuts de ses candidatures
Mes données	<code>MesDonnees.jsx</code>	Voir / supprimer ses données (RGPD)

Recruteur

Page	Fichier	Description
Dashboard	<code>Dashboard.jsx</code>	KPIs, alertes, top candidats
Candidats	<code>Candidates.jsx</code>	Liste de tous les candidats
Détail candidat	<code>CandidateDetail.jsx</code>	Profil complet + CV parsé
Offres	<code>OffersList.jsx</code>	Gestion des offres
Détail offre	<code>OfferDetail.jsx</code>	Candidats de l'offre + matching IA

Admin

Page	Fichier	Description
Gestion utilisateurs	<code>AdminUsers.jsx</code>	Liste, activation, suppression

Protection des routes

Chaque route est protégée — un candidat ne peut pas accéder aux pages recruteur et vice-versa.

```
// frontend/src/App.jsx
<ProtectedRoute allowedRoles={["recruteur", "admin"]} >
  <Dashboard />
</ProtectedRoute>
```

Communication avec le backend

Tous les appels API passent par `frontend/src/services/api.js`. Le token JWT est automatiquement ajouté à chaque requête.

```
// frontend/src/services/api.js
api.interceptors.request.use(config => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

Document généré le 12 mai 2026 — Projet de stage 3S TalentMatch